

Level 1 Comprehensive Master Lesson Plan (Days 1–24)

Course: 30-Day Python + VS Code + GitHub Builder Challenge (As-saadah Level 1)

Instructor: Syed Ahmed Kabir Hashmi

Overview: This document contains the complete detailed lesson plans, core concepts, build requirements, and starter templates for all 24 core classes of Level 1.

| Day 1: Environment Setup & Python Basics

Goal: Install VS Code, Python, and configure the local development environment and terminal workflow.

Core Concepts

- What is Python and how the interpreter executes code.
- Setting up VS Code extensions (Python, Pylance, GitLens).
- Running scripts via terminal (``python main.py``).

The Build

Write your first interactive greeting script that captures user input and prints a personalized welcome message.

| Day 2: Variables, Data Types & Basic I/O

Goal: Master variables, strings, integers, floats, booleans, and type casting.

Core Concepts

- Variable assignment and naming conventions (snake_case).
- Primitive data types and type conversion (``int()``, ``str()``, ``float()``).
- Using ``input()`` and formatted f-strings.

The Build

Build a "Tip Calculator & Splitter" that calculates individual shares based on bill total and tip percentage.

Day 3: Conditionals (`if`, `elif`, `else`)

Goal: Implement decision-making logic in Python scripts.

Core Concepts

- Comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`).
- Logical operators (`and`, `or`, `not`).
- Branching execution flow with `if`, `elif`, and `else`.

The Build

Build an "Interactive Ticket Pricing & Discount System" based on user age and membership status.

Day 4: Loops & Repeated Execution

Goal: Master `while` loops to keep programs running dynamically.

Core Concepts

- `while` loops vs `for` loops.
- Loop counters and state management (`attempts += 1`).
- Loop control statements (`break`, `continue`).

The Build

CLI Number Guessing Game with high/low feedback and attempt tracking.

```
secret_number = 7
attempts = 0
while True:
    guess = int(input("Guess (1-20): "))
    attempts += 1
    if guess == secret_number:
        print(f"Correct in {attempts} attempts!")
        break
    print("Too high!" if guess > secret_number else "Too low!")
```

| Day 5: For Loops & Iterables (Simple Task Manager)

Goal: Iterate over sequences and collections using ``for`` loops and ``range()``.

Core Concepts

- Iterating through lists and strings.
- Using ``range(start, stop, step)``.
- Accumulator patterns.

The Build

Console Task Manager that prints formatted task checklists and iterates through pending items.

| Day 6: Functions & Modular Code

Goal: Encapsulate reusable logic into custom functions with parameters and return values.

Core Concepts

- Defining functions with ``def``.
- Passing positional and keyword arguments.
- Using ``return`` values vs printing.

The Build

Modular Calculator App with dedicated functions for add, subtract, multiply, and divide.

| Day 7: Lists & Basic Data Management

Goal: Store, manipulate, and query ordered collections of data.

Core Concepts

- List creation, indexing, and slicing.
- Common methods: ``append()``, ``insert()``, ``remove()``, ``pop()``, ``sort()``.

The Build

Dynamic Shopping Cart application allowing users to add, remove, and view items.

| Day 8: Dictionaries & Key-Value Storage

Goal: Store structured data using key-value pairs.

Core Concepts

- Creating dictionaries and accessing values by keys.
- Methods: ``.keys()``, ``.values()``, ``.items()``, ``.get()``.

The Build

Student Grade Management System mapping student names to exam scores and subjects.

| Day 9: Tuples & Sets

Goal: Understand immutable sequences (tuples) and unique collections (sets).

Core Concepts

- Tuples for fixed records and multi-value returns.
- Sets for eliminating duplicates and performing set math (union, intersection).

The Build

Unique Tag Extractor for blog posts or user profiles using set operations.

| Day 10: Error Handling (``try`/`except``)

Goal: Prevent application crashes with robust exception handling.

Core Concepts

- Common exceptions (``ValueError``, ``TypeError``, ``ZeroDivisionError``, ``FileNotFoundError``).
- Using ``try``, ``except``, ``else``, and ``finally``.

The Build

Safe Division and Input Parser that gracefully recovers from invalid user input.

| Day 11: File Handling (Reading/Writing Text Files)

Goal: Persist data to local text and CSV files.

Core Concepts

- Using ``open()``, ``read()``, ``write()``, and ``close()``.
- Context managers (``with open(...)` as `f:``).

The Build

Persistent Journal App that appends daily reflections to a local ``journal.txt`` file.

| Day 12: Mini-Project 1: CLI Note-Taking App

Goal: Combine functions, file handling, loops, and lists into a complete application.

Core Concepts

- Modular application design.
- Loading state from disk on startup and saving on exit.

The Build

Fully functional CLI Note-Taking App with add, view, search, and delete features.

| Day 13: Introduction to OOP (Classes & Objects)

Goal: Model real-world entities using Object-Oriented Programming principles.

Core Concepts

- Classes as blueprints and objects as instances.
- The ``__init__`` constructor and ``self`` reference.

The Build

Bank Account simulation modeling customer accounts with deposit and withdrawal methods.

| Day 14: Methods, Constructors & Attributes

Goal: Build robust class methods and manage object state.

Core Concepts

- Instance attributes vs class attributes.
- Instance methods and object state modification.

The Build

Library Book Inventory system tracking book titles, authors, and checkout status.

| Day 15: Inheritance & Polymorphism

Goal: Extend existing classes and override methods for code reuse.

Core Concepts

- Base classes and derived subclasses.
- Using `super()` to initialize parent attributes.
- Method overriding.

The Build

Employee Management hierarchy with Base Employee, Manager, and Developer subclasses.

| Day 16: Git & GitHub Basics

Goal: Initialize local repositories, stage changes, commit, and push to GitHub.

Core Concepts

- Version control fundamentals.
- `git init`, `git add`, `git commit`, `git status`.
- Linking local repo to GitHub remote.

The Build

Create a GitHub repository for your Python project and push your daily challenge scripts.

| Day 17: Branching, Merging & Remote Repositories

Goal: Collaborate and manage feature branches in Git.

Core Concepts

- Creating and switching branches (`git checkout -b``).
- Merging branches (`git merge``).
- Handling basic merge conflicts.

The Build

Develop a new feature on a separate branch and merge it into `main`` before pushing to GitHub.

| Day 18: Mini-Project 2: CLI Contact Book with Persistence

Goal: Build an OOP-based contact book saved to disk with full Git version control.

Core Concepts

- Combining OOP classes (`Contact``, `ContactBook``) with file serialization.

The Build

CLI Contact Book application with search, update, delete, and file storage features managed via Git.

| Day 19: Working with Built-in Modules

Goal: Leverage Python standard library modules (`random``, `math``, `datetime``, `os``).

Core Concepts

- Importing modules (`import math``, `from datetime import datetime``).
- Exploring standard library utilities.

The Build

Password Generator and Date Calculator utility script.

| Day 20: Working with APIs & JSON (`requests`)

Goal: Fetch live data from web APIs using HTTP requests and JSON parsing.

Core Concepts

- HTTP GET requests with `requests.get()`.
- Parsing JSON responses into Python dictionaries.

The Build

Live Weather Fetcher or Joke Generator pulling data from a public API.

| Day 21: Environment Variables & Configuration

Goal: Securely manage API keys and configuration settings using `.env` files.

Core Concepts

- The `python-dotenv` library.
- Accessing environment variables via `os.environ`.
- Adding `.env` to `.gitignore`.

The Build

Configured API script loading secret credentials securely from a local environment file.

| Day 22: Database Integration (Introduction to SQLite)

Goal: Store and query structured relational data using SQLite.

Core Concepts

- Connecting to SQLite databases in Python (`sqlite3`).
- Executing SQL queries (`CREATE TABLE`, `INSERT`, `SELECT`).

The Build

Todo Application backed by a local SQLite database file.

| Day 23: Capstone Project Architecture & Planning

Goal: Design the architecture, schema, and feature scope for the Level 1 Capstone.

Core Concepts

- Software requirements specification (SRS).
- Defining classes, database schemas, and user interface flow.

The Build

Detailed Capstone Blueprint Document outlining features, data models, and milestones.

| Day 24: Capstone Build, Print & Final Submission

Goal: Complete, test, document, and publish your Capstone project to GitHub.

Core Concepts

- Writing professional `README.md` documentation.
- Final code cleanup, testing, and GitHub submission.

The Build

Completed Capstone Project fully documented and published on GitHub for Level 1 certification.